UniSpot

A Scalable Campus Events Platform — Project Report

Team 6: Mohamed Osama (16005523), Yassin Waleed (16006124), Saged Mohamed (16008234), Fares Sadek Galal (10002424), Hussein Sherif (13001338), Jumana Mohab (13007113), Mohamed Khalid Naguib (13001416) — CS

Course: ICS608 – Software Cloud Computing — Instructor: John Fayz

Load Balancer URL: http://UniSpot-ALB-909099899.eu-north-1.elb.amazonaws.com

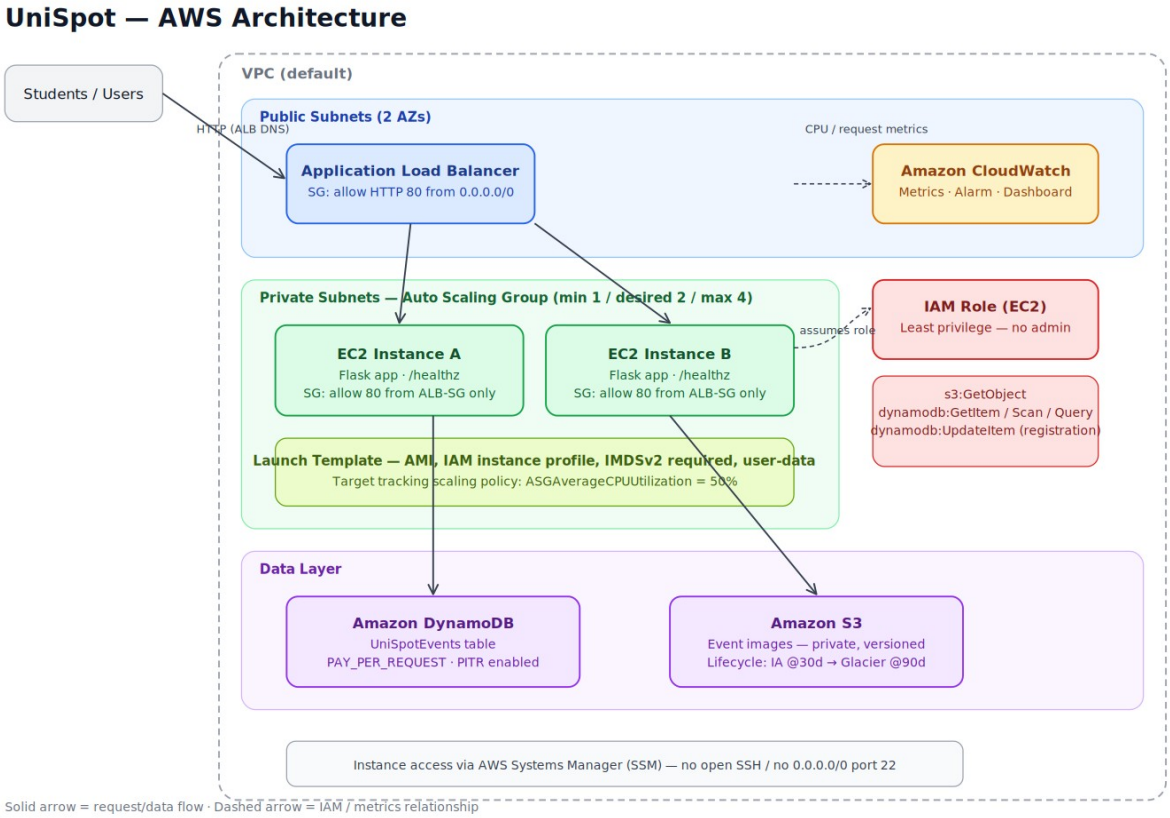
GitHub Repository: https://github.com/mohamedosama202/campus-events-unispot

1. Purpose of the Application

UniSpot is a cloud-based platform that helps university students discover campus events, club activities, workshops, sports tournaments, and social gatherings. Students can browse upcoming events, view event details, register interest, and filter events by category. While the application itself is intentionally simple, the primary objective of this project is to design and deploy a secure, scalable, highly available, and monitored AWS infrastructure.

2. Architecture Overview

The diagram below shows the full request path: users reach the application exclusively through the Application Load Balancer's DNS name, never an EC2 public IP. The Load Balancer distributes traffic across EC2 instances managed by an Auto Scaling Group. Application servers read event data from DynamoDB and event images from S3, assuming a least-privilege IAM role rather than embedding credentials. CloudWatch observes the whole stack and raises an alarm on sustained high CPU.



3. Responsibility of Each AWS Service

Service	Responsibility in UniSpot
Amazon EC2	Hosts the Flask application; each instance reports its own instance ID to demonstrate load distribution.
Application Load Balancer	Single entry point; distributes traffic, performs health checks on /healthz, stops routing to unhealthy targets.
Auto Scaling Group	Maintains 2 desired / 1 min / 4 max instances; replaces unhealthy instances automatically; scales on average CPU utilization.
Amazon DynamoDB	Stores event records (ID, name, category, date, location, spaces, registration count).

Amazon S3	Stores event poster images; private, versioned, with a lifecycle rule to move old versions to cheaper storage.
AWS IAM	Grants the EC2 role only the permissions it needs: read S3, read/update DynamoDB — no administrator access.
Security Groups	ALB SG accepts HTTP from the internet; EC2 SG accepts traffic only from the ALB SG; SSH is avoided in favor of SSM.
Amazon CloudWatch	Tracks CPU, request count, healthy/unhealthy host count and ASG activity; alarms and a dashboard surface issues.

4. Security Decisions

- EC2 instances run under a dedicated IAM role scoped to exactly the S3 and DynamoDB actions the app needs — no AdministratorAccess.
- The S3 bucket blocks all public access; the app reaches images through short-lived presigned URLs generated server-side.
- Security groups are layered: only the ALB SG accepts inbound traffic from the internet (port 80); the EC2 SG accepts traffic only from the ALB SG's ID, not from 0.0.0.0/0.
- Instance access uses AWS Systems Manager Session Manager instead of SSH, removing the need for an open port 22 or a distributed key pair. If SSH is enabled for debugging, it is restricted to a single trusted IP.
- IMDSv2 is enforced on all instances (HttpTokens: required) to reduce SSRF-based credential theft risk.
- No AWS credentials are stored in the application code or repository; the app receives temporary credentials via the EC2 instance profile.

5. Scaling and Availability Decisions

The Auto Scaling Group spans at least two Availability Zones behind the ALB, so the platform tolerates the loss of a single instance or AZ without downtime. The desired capacity of 2 keeps two servers running at all times for normal traffic; the group can scale out to 4 instances during traffic spikes (e.g. a major tournament announcement) using a target-tracking policy on average CPU utilization (target 50%), and back in once demand subsides. Unhealthy instances — detected via ALB health checks against /healthz — are automatically terminated and replaced.

6. Monitoring Configuration

A CloudWatch dashboard tracks four metrics: EC2 average CPU utilization, ALB request count, ALB healthy/unhealthy host count, and Auto Scaling Group in-service instance count. A CloudWatch alarm fires when average CPU exceeds 70% for two consecutive 5-minute periods; it can optionally notify the team by email through an SNS topic.

7. Challenges Faced and How They Were Solved

The most significant challenge occurred after deployment: event images uploaded to S3 were returning "AccessDenied" errors in the browser even though the S3 bucket, IAM permissions, and application code for generating presigned URLs were all configured correctly. Investigation showed the boto3 S3 client was defaulting to the global endpoint (s3.amazonaws.com) when generating presigned URLs, while the signature was scoped to the eu-north-1 region — a mismatch that AWS rejects for any region other than us-east-1. The fix was to explicitly set a regional endpoint_url (https://s3.eu-north-1.amazonaws.com) on the boto3 S3 client. After redeploying the corrected code to the deployment S3 bucket and running an Auto Scaling Group instance refresh (which replaced running instances with zero downtime, since the ALB kept routing traffic to the remaining healthy instance throughout), all event images loaded correctly. A second, smaller issue arose during testing: the target-tracking CPU scaling policy repeatedly scaled the group back down to 1 instance during idle periods, which is correct cost-saving behavior but made it harder to reliably demonstrate two-instance load balancing on demand. This was resolved by temporarily

setting the minimum capacity to 2 while capturing verification evidence, then returning it to the original minimum afterward.

8. Cost Estimate

Official estimate generated using AWS Pricing Calculator for the eu-north-1 (Stockholm) region:
https://calculator.aws/#/estimate?id=e1ae0f302172dd1d68c5e47f4b350b55851e1dde

AWS Component	Configuration	Est. Monthly Cost (USD)
Amazon EC2	2x t3.micro, on-demand, eu-north-1	$15.77
Elastic Load Balancing (ALB)	1 Application Load Balancer	$17.70
Amazon DynamoDB	On-demand capacity mode	$0.27
Amazon S3	Standard storage, ~1GB, low request volume	$0.02
Amazon CloudWatch	1 dashboard, 1 alarm, standard metrics	$1.50
Total		**$35.26 / month**

This estimate reflects steady-state usage (both EC2 instances running continuously, low traffic volume). Full estimate details and line-item breakdowns are available at the calculator.aws link above.

9. Team Contributions

Each team member owned a specific part of the codebase and infrastructure, with close collaboration throughout on design decisions, testing, and documentation.

Team Member	Contribution
Mohamed Osama	Flask backend (app.py: routes, DynamoDB and S3 integration, health check); project report.
Yassin Waleed & Saged Mohamed	Database seeding (seed_data.py) and local application testing (test_local.py).
Fares Sadek Galal & Hussein Sherif	AWS infrastructure setup and configuration (EC2, ALB, Auto Scaling Group, CloudFormation template, server startup script).
Jumana Mohab	IAM roles and policies, ensuring least-privilege access for the EC2 instance role.
Mohamed Khalid Naguib	Frontend templates (homepage, event detail pages) and static styling.

10. Bonus Features Implemented

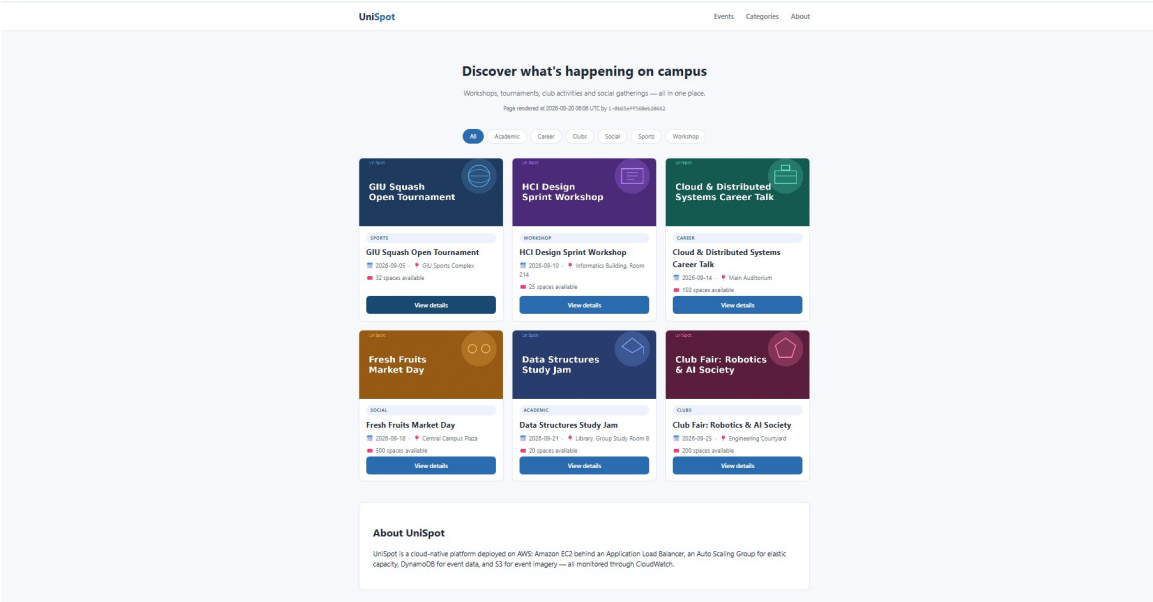
Beyond the required components, the following optional bonus features from the assignment brief were implemented and verified:

- Event search and category filters: the homepage includes clickable category chips (All, Academic, Career, Clubs, Social, Sports, Workshop) that filter the visible events client-side without a page reload.
- DynamoDB Point-in-Time Recovery: continuous backups with PITR are enabled on the UniSpotEvents table (35-day recovery window), verified via aws dynamodb describe-continuous-backups.
- SNS notification connected to the CloudWatch alarm: an SNS topic (UniSpot-Alerts) is subscribed to the team's email and attached as an alarm action on UniSpot-HighCPU, so the team is notified automatically if CPU exceeds the 70% threshold.
- Responsive interface for mobile devices: dedicated CSS media queries (max-width 640px and 380px) reflow the layout for phone-sized screens — the event grid collapses to a single column, buttons become full-width, and header/hero text sizes adjust — verified in both desktop and mobile viewport widths.

Appendix A: Verification Screenshots

The following screenshots and command outputs were captured during live testing of the deployed stack, mapped to the required demonstration items from the assignment brief.

1-4. Homepage via ALB DNS, event data from DynamoDB, images from S3



Homepage loaded via the Application Load Balancer DNS name, showing all 6 seeded DynamoDB events with images served from S3 via presigned URLs, and the responding instance ID in the footer.

2. Instance ID rotation across requests (ALB load balancing)

Ten sequential requests to the ALB DNS name, each opening a fresh connection:

```
$ for i in {1..10}; do curl -s http://UniSpot-ALB-909099899.eu-north-
1.elb.amazonaws.com \
    | grep -oP 'by <code>\K[^<]+'; done
i-0ac58f27c08a24fc6
i-0ac58f27c08a24fc6
i-0b65eff560eb20662
i-0ac58f27c08a24fc6
i-0b65eff560eb20662
i-0b65eff560eb20662
i-0b65eff560eb20662
i-0b65eff560eb20662
i-0ac58f27c08a24fc6
i-0b65eff560eb20662
```

Requests were distributed across both registered EC2 instances, confirming the ALB is load balancing correctly.

3. Target group health check

```
$ aws elbv2 describe-target-health --target-group-arn <UniSpot-TG ARN>
{"TargetHealthDescriptions": [
  {"Target": {"Id": "i-0ac58f27c08a24fc6"}, "TargetHealth": {"State": "healthy"}},
  {"Target": {"Id": "i-0b65eff560eb20662"}, "TargetHealth": {"State": "healthy"}}
]}
```

5-7. Instance failure tolerance and Auto Scaling Group self-healing

One EC2 instance was manually stopped via the AWS CLI to simulate a failure:

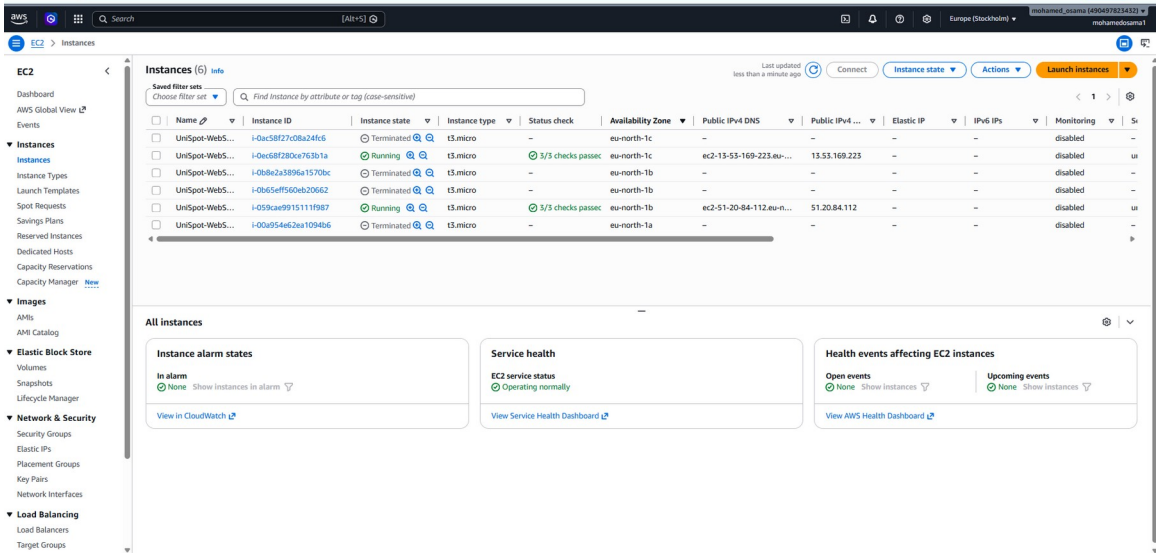
```
$ aws ec2 stop-instances --instance-ids i-0ac58f27c08a24fc6
"CurrentState": {"Name": "stopping"}, "PreviousState": {"Name": "running"}
```

The application remained fully available, served entirely by the remaining healthy instance:

```
$ for i in {1..5}; do curl -s http://UniSpot-ALB-.../ | grep -oP 'by <code>\K[^<]
+'; done
i-0b65eff560eb20662
i-0b65eff560eb20662
i-0b65eff560eb20662
```

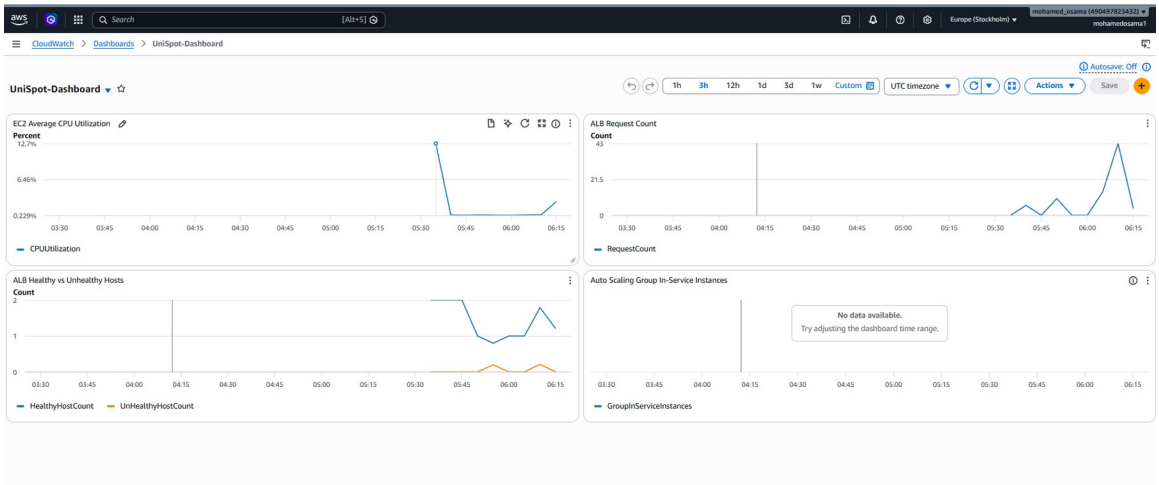
The Auto Scaling Group detected the unhealthy instance and automatically launched a replacement:

```
Health     Instance ID             State
Unhealthy  i-0ac58f27c08a24fc6     Terminating
Healthy    i-0b65eff560eb20662     InService
Healthy    i-0ec68f280ce763b1a     InService   <- new replacement instance
```

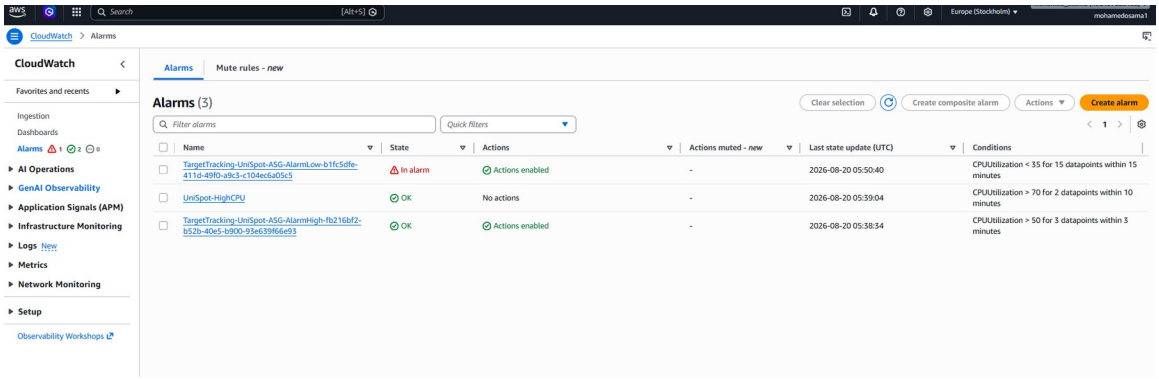


EC2 console instance history showing multiple terminated instances from testing (stop/replace cycles and instance refreshes) alongside the current two running, healthy instances.

8. CloudWatch Dashboard and Alarm

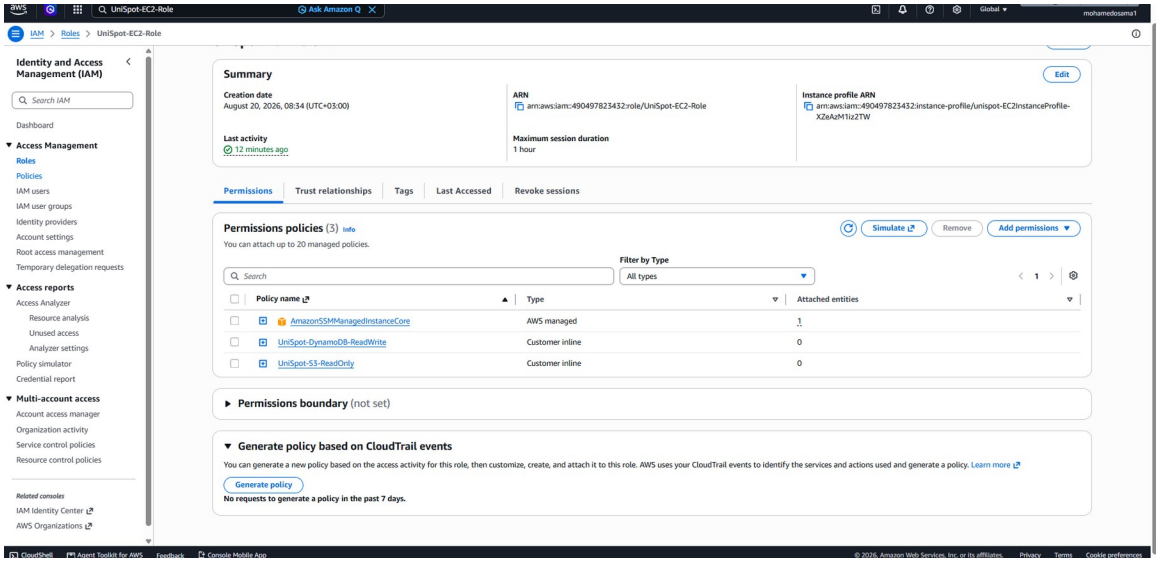


UniSpot-Dashboard: EC2 CPU utilization, ALB request count, and ALB healthy vs. unhealthy host count (the dip and recovery visible here corresponds directly to the instance-stop test above).

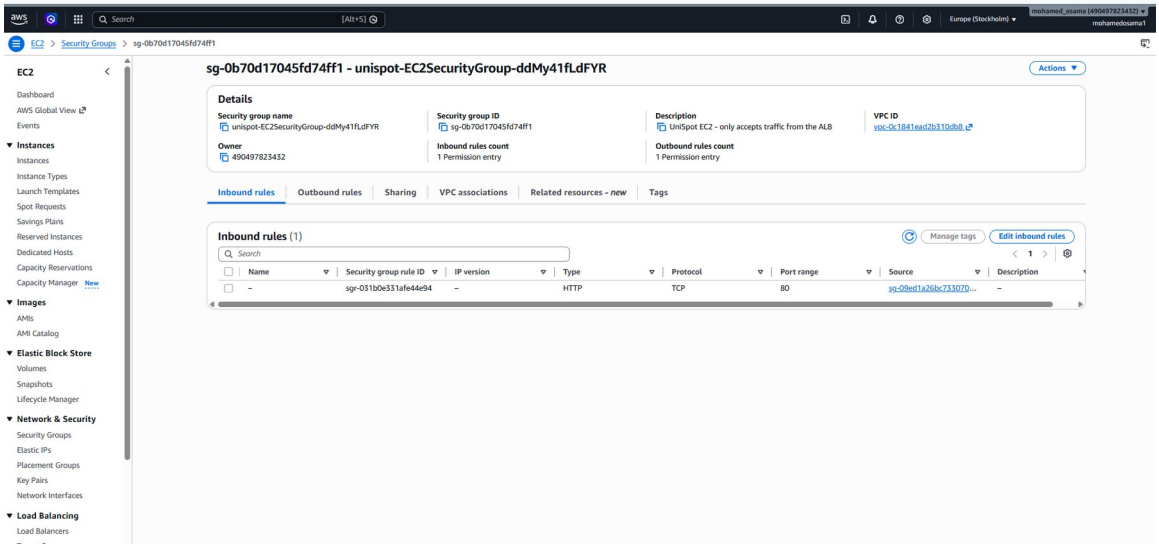


CloudWatch Alarms: UniSpot-HighCPU is in OK state (no sustained high load). The target-tracking scaling alarms are also visible, responsible for automatic scale-in/scale-out.

9. IAM Role and Security Group Configuration

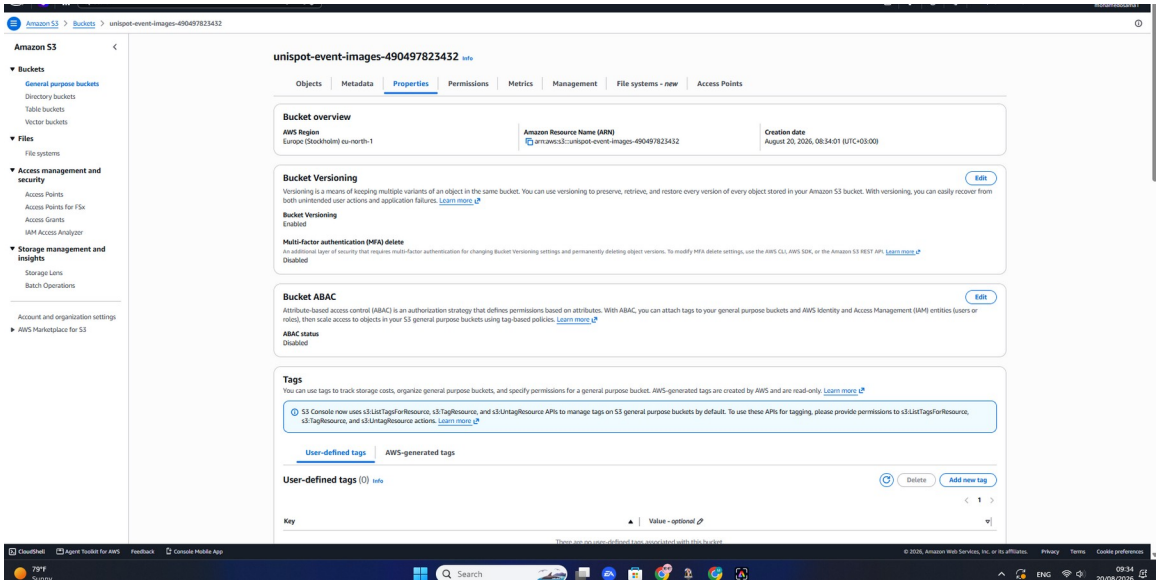


UniSpot-EC2-Role permissions: only AmazonSSMManagedInstanceCore, a scoped DynamoDB read/write inline policy, and a scoped S3 read-only inline policy. No administrator access.

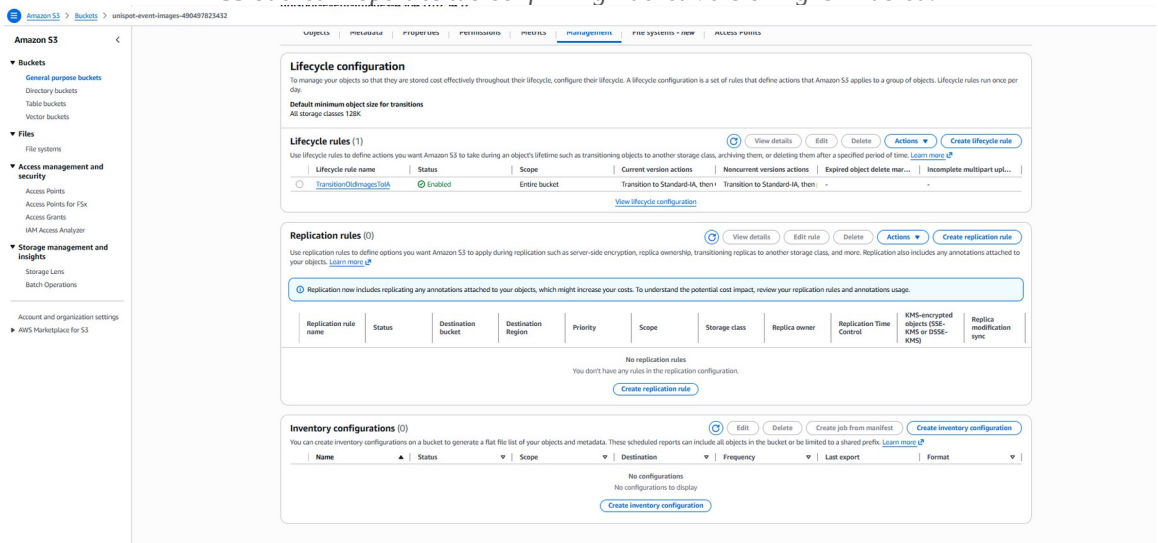


EC2 security group inbound rules: HTTP (port 80) permitted only from the ALB's security group ID, not from 0.0.0.0/0 — EC2 instances are unreachable directly from the internet.

10. S3 Bucket Versioning and Lifecycle Configuration



S3 bucket Properties tab confirming Bucket Versioning is Enabled.



S3 Management tab showing the active lifecycle rule (TransitionOldImagesToIA), transitioning older object versions to a cheaper storage class.